

基于元学习的深度在线学习：面向基于模型强化学习的持续自适应

Anusha Nagabandi, Chelsea Finn, Sergey Levine
{nagaban2, cbfinn, sergey.levine}@berkeley.edu
UC Berkeley

摘要

人类和动物能够学习复杂的预测模型，从而准确可靠地推理现实世界现象，并在面对意外变化时极快地调整这些模型。深度神经网络模型使我们能够表示非常复杂的函数，但缺乏这种快速在线适应的能力。本文的目标是开发一种方法，利用深度神经网络模型，从连续到达的数据流中进行持续在线学习。我们提出了一种在线学习过程，该过程使用随机梯度下降来更新模型参数，并使用带有中餐馆过程先验的期望最大化算法来开发和维护模型混合体，以处理非平稳任务分布。这使得所有模型都能按需调整，在任务变化时实例化新模型，并在再次遇到先前见过的任务时召回旧模型。此外，我们观察到元学习可以用来训练一个模型，使得这种使用随机梯度下降的直接在线适应是有效的，而对于大型函数逼近器来说，否则并非如此。在这项工作中，我们将用于在线学习的元学习（MOLe）方法应用于基于模型的强化学习，其中调整预测模型对于控制至关重要；我们证明 MOLe 优于其他先前方法，并能在非平稳任务分布（如变化的地形、电机故障和意外干扰）中实现有效的持续适应。视频可在以下网址获取：
<https://sites.google.com/berkeley.edu/onlineviameta>

1 引言

人类和动物的学习不仅体现在获取复杂技能的能力上，还体现在当这些技能必须在新条件或变化条件下执行时能够快速适应的能力。例如，动物可以快速适应在不同表面上行走和奔跑（Herman, 2017），人类可以在存在意外扰动的情况下轻松调节伸手动作中的力（Flanagan & Wing, 1993）。此外，这些经验会被记住，并且可以在未来出现类似干扰时被回忆起来以更快地适应（Doyon & Benali, 2005）。由于在如此短的时间尺度上学习全新的模型是不切实际的，我们可以设计算法，明确训练模型从少量数据中快速适应。这种在线适应对于在现实世界中运行的智能系统至关重要，在现实世界中，变化因素和意外扰动是常态。在本文中，我们提出了一种用于快速连续在线学习的算法，该算法利用深度神经网络模型来构建和维护任务分布，从而自然地发展出泛化能力和任务专业化。

我们的工作示例是基于模型的强化学习环境中的连续适应，尽管我们的方法通常适用于任何具有流数据的在线学习场景。我们假设每个“试验”包含多个任务，并且任务之间的划分不会明确提供给学习者——相反，该方法必须自适应地决定“任务”代表什么、何时实例化新任务以及何时继续更新旧任务。例如，在变化地形上运行的机器人可能需要处理上坡和下坡，并且可能

选择维护分别针对每个坡度专业化的独立模型，根据当前推断的表面依次适应每个模型。

我们通过在线随机梯度下降（SGD）对模型参数进行简单适应，同时为不同任务维护模型参数的混合模型。该混合模型通过中国餐馆过程（Stimberg et al., 2012）更新，从而在试验过程中根据需要实例化新任务。尽管在线学习可能是 SGD 最古老的应用之一（Bottou, 1998），但现代参数模型如深度神经网络用这种方法在线训练极其困难。它们通常需要中等规模的小批量和多个训练轮次才能得到合理解，这在在线流式接收数据时并不合适。我们的一个关键观察是，元学习可用于学习参数的先验初始化，使得这种直接在线适应只需少量梯度步骤即可实现。我们使用的元训练过程基于模型无关元学习（MAML）（Finn et al., 2017），其中学习模型的先验权重初始化，以在少量梯度步骤后优化来自元训练任务分布的任何任务的改进。

基于 MAML 的元学习此前已扩展到基于模型的强化学习（Nagabandi et al., 2018），但仅适用于 k -次适应设置：元学习的先验模型适应于 k 个最近时间步，但适应不会随时间向前传递（即适应总是从先验本身开始）。这种僵化的批处理模式设置在线学习环境中具有限制性，并且对于训练分布之外的任务不够充分。更自然的表述是模型接收连续数据流，并必须在线适应潜在非平稳的任务分布。这既需要快速适应和回忆先前任务的能力，也需要有效的适应策略以在两者之间按需插值。

本文的主要贡献是一种用于在线学习的元学习（MOLe）算法，该算法利用期望最大化，并结合任务分布上的中餐馆过程先验，来学习神经网络模型的混合体，每个模型都通过在线随机梯度下降进行更新。与以往的多任务和元学习方法相比，本方法在线分配任务概率，使任务专业化能够自然涌现，无需预先指定任务划分。我们在基于模型的强化学习背景下评估了 MOLe，测试环境包括一系列具有挑战性的模拟机器人任务，涵盖干扰、环境变化和模拟电机故障。模拟实验显示，半猎豹智能体和六足爬行机器人在在线环境中进行连续模型自适应。结果表明，MOLe 能够在线实例化新任务，适应分布外任务，并识别和回退到先前任务。此外，我们证明 MOLe 优于最先进的先前方法，该方法执行 k 次基于模型的元强化学习，以及自然基线，如用于自适应的连续梯度更新和未经元训练的在线学习。

2 相关工作

在线学习是机器学习最古老的子领域之一（Bottou, 1998; Jafari et al., 2001）。先前算法使用了在线梯度更新（Duchi et al., 2011）和概率滤波公式（Murphy, 2002; Hoffman et al., 2010; Broderick et al., 2013）。原则上，常用的基于梯度的学习方法，如随机梯度下降，可以很容易地用作在线学习算法（Bottou, 1998）。然而，在实践中，它们在深度神经网络函数逼近器上的性能有限（Sahoo et al., 2017）：此类高维模型必须使用批处理模式方法、小批量和多次遍历数据进行训练。我们旨在通过使用模型无关元学习（MAML）来显式预训练一个支持快速自适应的模型，从而解除这一限制，然后通过期望最大化算法与中餐馆过程（Blei et al., 2003）先验相结合，在非平稳任务分布中动态分配新任务，实现连续在线自适应。

在线学习与持续学习或终身学习（Lifelong Learning）（Thrun, 1998）相关，其中智能体随时间面对非平稳的任务分布。然而，与专注于避免负迁移（即灾难性遗忘（Catastrophic Forgetting））的工作（Kirkpatrick et al., 2017; Rebuffi et al., 2017; Zenke et al., 2017; Lopez-Paz et al., 2017; Nguyen et al., 2017）不同，在线学习侧重于在非平稳情况下快速学习和适应的能力。虽然一些持续学习工作

考虑了前向迁移问题，例如 Rusu et al. (2016); Aljundi et al. (2017); Wang et al. (2017)，但这些工作以及持续学习中的其他工作通常关注小规模任务集，在这些任务集中，快速在线学习实际上不可行，因为根本没有足够的任务来恢复结构，从而无法在新任务或新环境中实现快速的小样本学习（Few-Shot Learning）。

本文方法建立在元学习（Meta-Learning）或学会学习（Learning-to-Learn）技术之上（Thrun & Pratt, 1998; Schmidhuber, 1987; Bengio et al., 1992; Naik & Mammone, 1992）。然而，近期大多数元学习研究考虑的是每次学习一个任务的设定，且通常基于单批数据（Santoro et al., 2016; Ravi & Larochelle, 2017; Munkhdalai & Yu, 2017; Wang et al., 2016; Duan et al., 2016）。本文特别针对非平稳任务分布，且不假设任务边界已知。先前工作（Jerfel et al., 2018）也考虑了非平稳任务分布；但 Jerfel 等人（2018）利用元梯度估计任务特定参数混合模型的参数，而本文关注在运行时优化过程中快速适应并累积任务特定的混合成分。其他元学习研究考虑了任务内部的非平稳性（Al-Shedivat et al., 2017）以及元测试阶段涉及多任务的片段（Ritter et al., 2018），但未考虑任务分离未知情况下的持续在线适应。先前工作还研究了基于模型的强化学习（Model-Based RL）的元学习（Nagabandi et al., 2018）。该先前方法每个时间步更新模型，但每次更新均为批量模式 k -样本更新，恰好使用 k 条先前转移并在每一步重置模型。这允许自适应控制，但无法实现持续在线适应，因为先前步骤的更新总是被丢弃。在比较中，本文发现所提方法显著优于该先前方法。据我们所知，本文工作是首次将元学习应用于学习流式在线更新。

3 问题陈述

我们将在线学习问题形式化如下：在每个时间步，模型接收输入 $\mathbf{x}_t$ 并产生预测 $\hat{\mathbf{y}}_t$ 。随后它接收真实标签 $\mathbf{y}_t$ ，必须利用该标签调整模型，以提高对下一个输入 $\mathbf{x}_{t+1}$ 的预测准确率。假设真实标签来自某个任务分布 $P(\mathbf{y}_t | \mathbf{x}_t, T_t)$ ，其中 T_t 是时间 t 处的任务。任务本身随时间变化，导致非平稳任务分布，且任务 T_t 的身份对学习未知。在现实场景中，任务可能对应系统的未知参数（例如机器人上的电机故障）、用户偏好或其他意外事件。该问题陈述涵盖了一系列在线学习问题，这些问题都需要对流式数据进行持续适应，并在泛化与专精之间进行权衡。

在实验中，我们以基于模型的强化学习作为工作示例，其中输入 $\mathbf{x}_t$ 是状态-动作对，输出 $\mathbf{y}_t$ 是下一状态。我们在第 6 节讨论该方法在基于模型的强化学习中的应用，但以下推导保持通用性，适用于任意在线预测问题。

4 基于元训练网络混合模型的在线学习

我们分两部分讨论面向在线学习的元学习（MOLe）算法：本节讨论在线学习，下一节讨论元学习。本节阐述一种在线学习方法，能够利用来自非平稳任务分布的连续数据流进行有效在线学习。我们旨在保留泛化能力以免丢失过去知识，同时获得专精能力，这对于学习分布外程度更高、需要更多学习的新任务尤为重要。我们在第 5 节讨论获取元学习先验的过程，但本节首先形式化一种使用随机梯度下降与期望最大化的在线适应算法，以维护和调整任务模型参数上的混合模型（即概率任务分布）。

4.1 方法概述

令 $p_{\theta(T_t)}(\mathbf{y}_t | \mathbf{x}_t)$ 表示模型在输入 $\mathbf{x}_t$ 上对未知任务 T_t 的预测分布。我们的目标是估计非平稳任务分布中每个任务 T 的模型参数 $\theta_t(T)$ ：这需要在每一步推断任务上的分布 $P(T_t | \mathbf{x}_t, \mathbf{y}_t)$ ，利用该分布进行预测 $\hat{\mathbf{y}}_t = p_{\theta(T_t)}(\mathbf{y}_t | \mathbf{x}_t)$ ，并利用它更新每个模型

$\theta_t(T)$ 至 $\theta_{t+1}(T)$ 。在实践中，每个模型的参数 $\theta(T)$ 对应于神经网络 $f_{\theta(T)}$ 的权重

每个模型以某个先验参数向量 θ^* 开始，我们将在第 5 节中更详细地讨论。由于任务数量也是未知的，我们在时间步 0 时从一个任务开始，其中 $\theta_0(T) = \{\theta_0(T_0)\} = \{\theta^*\}$ 。从这里开始，我们持续更新 $\theta_t(T)$ 中的所有参数，并根据需要添加新任务，以尝试对真实底层过程 $P(Y_t|X_t, T_t)$ 进行建模。由于任务身份未知，我们还必须在每个时间步估计 $P(T_t)$ 。因此，在线学习问题包括根据推断的任务概率 $P(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t)$ 在每个时间步 t 调整每个 $\theta_t(T_i)$ 。为此，我们采用期望最大化 (EM) 算法并优化期望对数似然，其表达式为

$$\mathcal{L} = E_{T_t \sim P(T_t|\mathbf{x}_t, \mathbf{y}_t)} [\log p_{\theta_t(T_t)}(\mathbf{y}_t|\mathbf{x}_t, T_t)], \quad (1)$$

其中我们使用 $\theta_t(T_i)$ 表示对应于任务 T_i 的模型参数。最后，为了处理未知的任务数量，我们采用中国餐馆过程来根据需要实例化新任务。

4.2 近似在线推断

我们使用期望最大化 (EM) 来更新模型参数。在我们的情况中，EM 中的 E 步骤涉及在当前时间步估计任务分布 $P(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t)$ ，而 M 步骤涉及从 $\theta_t(T)$ 更新所有模型参数以获得新的模型参数 $\theta_{t+1}(T)$ 。参数始终根据推断的任务责任在每个时间步通过一步梯度进行更新。

我们首先估计任务分布中所有任务参数 T_i 的期望，其中每个任务概率的后验可以写成如下形式：

$$P(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t) \propto p_{\theta_t(T_i)}(\mathbf{y}_t|\mathbf{x}_t, T_t = T_i)P(T_t = T_i). \quad (2)$$

然后，我们使用中国餐馆过程 (CRP) 来构建任务先验 $P(T_t = T_i)$ ，以便在试验期间实例化新任务。CRP 是狄利克雷过程的一个实例。在 CRP 中，在时间 t ，每个任务 T_i 的概率应由下式给出

$$P(T_t = T_i) = \frac{n_{T_i}}{t - 1 + \alpha} \quad (3)$$

其中 n_{T_i} 是任务 T_i 在所有步骤 $1, \dots, t-1$ 中的期望数据点数量， α 是控制新任务实例化的超参数。因此，先验变为

$$P(T_t = T_i) = \frac{\sum_{t'=1}^{t-1} P(T_{t'} = T_i)}{t - 1 + \alpha} \quad \text{and} \quad P(T_t = T_{\text{new}}) = \frac{\alpha}{t - 1 + \alpha} \quad (4)$$

结合先验与似然，我们推导出如下后验任务概率分布：

$$P(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t) \propto p_{\theta_t(T_i)}(\mathbf{y}_t|\mathbf{x}_t, T_t = T_i) \left[\sum_{t'=1}^{t-1} P(T_{t'} = T_i) + \delta(T_{t'} = T_{\text{new}})\alpha \right] \quad (5)$$

在估计出潜在任务概率之后，我们接下来执行 M 步骤，该步骤基于推断出的任务分布来提升方程 1 中的期望对数似然。由于每个任务都从先验 θ^* 开始，经过一次梯度更新后， $\theta_t(T)$ 中所有参数的值由下式给出

$$\theta_{t+1}(T_i) = \theta^* - \beta \sum_{t'=0}^t P_t(T_{t'} = T_i|\mathbf{x}_{t'}, \mathbf{y}_{t'}) \nabla_{\theta_{t'}(T_i)} \log p_{\theta_{t'}(T_i)}(\mathbf{y}_{t'}|\mathbf{x}_{t'}) \quad \forall T_i \quad (6)$$

如果我们假设 $\theta_t(T)$ 的所有参数已经针对之前的时间步 $0, \dots, t$ 进行了更新，那么我们可以通过仅在新数据上更新所有参数 $\theta_t(T)$ 来近似这一更新：

$$\theta_{t+1}(T_i) = \theta_t(T_i) - \beta P_t(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t) \nabla_{\theta_t(T_i)} \log p_{\theta_t(T_i)}(\mathbf{y}_t|\mathbf{x}_t) \quad \forall T_i \quad (7)$$

这一过程是一种近似，因为对任务参数 $\theta_t(T)$ 的更新实际上也会改变之前时间步上对应的任务概率。然而，这种近似

消除了存储先前观测数据点的需要，并产生了一个完全在线、流式的算法。最后，为了完整实现 EM 算法，我们必须在每个时间步交替执行 E 步骤和 M 步骤直至收敛，并在每次迭代时将之前的梯度更新回滚到 $\theta_t(T)$ 。在实践中，我们发现每个时间步仅执行一次 E 步骤和 M 步骤就足够了。虽然这是一种粗略的简化，但在在线学习场景中，连续的时间步很可能是相关的，这使得该过程是合理的。不过，在保持完全在线的同时执行多步 EM 也是直接可行的。

我们现在总结 MOLe 的这一完整在线学习部分，并在算法 1 中给出概要。在第一个时间步 $t = 0$ ，任务分布被初始化为包含一个条目： $\theta_0(T) = \{\theta_0(T_0)\} = \{\theta^*\}$ 。在此之后的每个时间步，执行 E 步骤来估计任务分布，并执行 M 步骤来更新模型参数。CRP 先验还在每个时间步赋予在该时间步添加新任务 T_{new} 的概率。这个新任务的参数 $\theta_{t+1}(T_{\text{new}})$ 根据最新数据从 θ^* 进行适配。然后使用与最可能任务 T^* 对应的模型参数 $\theta_{t+1}(T^*)$ 对下一个数据点进行预测。

Algorithm 1 Online Learning with Mixture of Meta-Trained Networks

Require: θ^* from meta-training
Initialize $t = 0, \theta_0(T) = \{\theta_0(T_0)\} = \{\theta^*\}$
for each time step t **do**
 Calculate $p_{\theta_t(T_i)}(\mathbf{y}_t|\mathbf{x}_t, T_t = T_i) \forall T_i$
 Calculate $P_t(T_i) = P_t(T_t = T_i|\mathbf{x}_t, \mathbf{y}_t) \forall T_i$
 Calculate $\theta_{t+1}(T_i)$ by adapting from $\theta_t(T_i) \forall T_i$
 Calculate $\theta_{t+1}(T_{\text{new}})$ by adapting from θ^*
 if $P_t(T_{\text{new}}) > P_t(T_i) \forall T_i$ **then**
 Add $\theta_{t+1}(T_{\text{new}})$ to $\theta_{t+1}(T)$
 Recalculate $P_t(T_i)$ using $\theta_{t+1}(T_i) \forall T_i$
 Recalculate $\theta_{t+1}(T_i)$ using updated $P_t(T_i) \forall T_i$
 end if
 $T^* = \operatorname{argmax}_{T_i} p_{\theta_{t+1}(T_i)}(\mathbf{y}_t|\mathbf{x}_t, T_t = T_i)$
 Receive next data point $\mathbf{x}_{t+1}$
end for

5 元学习先验

上文提出了一种利用持续输入数据进行在线适应的算法。对于该方法，我们选择使用模型无关元学习 (MAML) 算法对先验进行元训练。该元训练算法是一个合适的选择，因为它产生的先验专门用于基于梯度的微调。在进一步讨论元训练过程的选择之前，我们首先概述 MAML 和一般的元学习。

给定任务分布，元学习算法产生一个学习过程，在某些情况下，该过程能够快速适应新任务。MAML 优化深度网络的初始化，使得在使用该任务的少量数据点进行微调时，能够实现良好的少样本任务泛化。在训练时，MAML 看到来自大量任务的小量数据，其中每个任务 T 的数据 $\mathcal{D}_T$ 可以划分为训练子集和验证子集 ($\mathcal{D}_T^{\text{tr}}$ 和 $\mathcal{D}_T^{\text{val}}$)，其中 $\mathcal{D}_T^{\text{tr}}$ 的大小为 k 。MAML 优化模型参数 θ ，使得在 $\mathcal{D}_T^{\text{tr}}$ 上进行一步或多步梯度下降后，在 $\mathcal{D}_T^{\text{val}}$ 上的损失 L 最小。在我们的情况中，我们设置 $\mathcal{D}_{T_t}^{\text{tr}} = (\mathbf{x}_t, \mathbf{y}_t)$ 和 $\mathcal{D}_{T_t}^{\text{val}} = (\mathbf{x}_{t+1}, \mathbf{y}_{t+1})$ ，损失 L 对应于负对数似然。一个良好的 θ 能够在各种元训练任务中成功实现这种适应，因此是一个良好的网络初始化，从该初始化出发，适应过程可以解决与先前见过的任务相关的各种新任务。MAML 目标定义如下：

$$\min_{\theta} \sum_T L(\theta - \eta \nabla_{\theta} L(\theta, \mathcal{D}_T^{\text{tr}}), \mathcal{D}_T^{\text{val}}) = \min_{\theta} \sum_T L(\phi_T, \mathcal{D}_T^{\text{val}}). \quad (8)$$

这里， η 是内部学习率。一旦该元目标被优化，得到的 θ^* 就充当先验，在测试时可以利用来自 $\mathcal{D}^{\text{tr}}$ 的近期经验进行微调，如下所示：

$$\phi_{T_{\text{test}}} = \theta^* - \eta \nabla_{\theta^*} L(\theta^*, \mathcal{D}_{T_{\text{test}}}^{\text{tr}}). \quad (9)$$

这里， $\phi_{T_{\text{test}}}$ 从元学习的先验 θ^* 适应而来，以更好地代表当前时刻。尽管 Finn 等人 (2017) 展示了深度神经网络的这种快速适应，Nagabandi 等人 (2018) 将该框架扩展到基于模型的元强化学习，但这些方法处理的是 k 样本设置下的适应，总是直接从元学习的先验进行适应，不允许进一步的适应或专门化。在本工作中，我们通过时间扩展的在线适应过程，扩展了这些能力，使知识能够更持续地演化。

虽然我们的持续在线学习过程仍然以这种用于 k 样本适应的元训练 (即 MAML) 进行初始化，但我们发现该先验足以实现有效的持续学习。

在测试时进行在线自适应。其直观依据在于，MAML 训练模型使其仅用少量数据点和梯度步就能产生显著变化。注意，这个元训练先验在测试时可用于 (a) k -样本设置，类似于其训练方式；也可在测试时通过 (b) 从该先验出发采取显著更多的梯度步来使用。我们在第 7 节中表明，本文方法优于这两种方法，但仅凭能够以这些方式使用该元学习先验，就使 MAML 的使用颇具吸引力。

我们注意到，完全可以修改 MAML 算法，使其直接针对第 4.2 节讨论的加权更新来优化模型。这只需在元训练期间对每个批次计算任务权重 (E 步)，然后构建一个计算图，其中所有梯度更新都乘以各自权重。标准自动微分软件随后可计算相应的元梯度。对于较短的试验长度，这并不比标准 MAML 复杂多少；对于较长的试验长度，截断反向传播是一种选择。尽管这种元训练过程更符合模型在在线自适应期间的使用方式，但我们发现它并未显著改善结果。虽然如果针对长期自适应进行元训练，差异可能更显著，但这一观察确实表明，仅用 MAML 进行元训练就足以在非平稳多任务环境中实现有效的持续在线自适应。需要澄清的是，尽管这种在元训练期间纳入 EM 权重更新的修改训练过程并未明确改善结果，但我们确实看到，使用更多数据进行标准 MAML 元训练过程时，测试时性能有所提升 (见附录)。

6 应用于基于模型的强化学习

在实验中，我们将 MOLe 应用于基于模型的强化学习。强化学习总体目标是采取行动以最大化未来奖励之和。在每个时间步 t ，智能体从状态 $\mathbf{s}_t \in S$ 执行动作 $\mathbf{a}_t \in A$ ，根据转移概率 (即动力学) $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ 转移到下一状态 $\mathbf{s}_{t+1}$ ，并获得奖励 $r_t = r(\mathbf{s}_t, \mathbf{a}_t)$ 。每一步的目标是执行动作 $\mathbf{a}_t$ ，以最大化未来奖励的折扣和 $\sum_{t'=t}^{\infty} \gamma^{t'-t} r_{t'}$ ，其中折扣因子 $\gamma \in [0, 1]$ 优先考虑近期奖励。特别是在基于模型的强化学习中，来自已知或学习到的动力学模型的预测被用于学习策略，或直接用于规划算法中以选择最大化奖励的动作。

在我们的工作中，我们旨在建模的底层分布是动力学分布 $p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t, T_t)$ ，其中未知的 T_t 代表底层设置 (例如，系统状态、外部细节、环境扰动等)。MOLe 的目标是用预测模型 p_θ 估计该分布。为了在基于模型的强化学习背景下实例化 MOLe，我们按照算法 1 进行，并采用以下具体说明：(1) 我们将输入 $\mathbf{x}_t$ 设置为 K 个先前状态和动作的拼接，由 $\mathbf{x}_t = [\mathbf{s}_{t-K}, \mathbf{a}_{t-K}, \dots, \mathbf{s}_{t-2}, \mathbf{a}_{t-2}, \mathbf{s}_{t-1}, \mathbf{a}_{t-1}]$ 给出，输出为相应的下一状态 $\mathbf{y}_t = [\mathbf{s}_{t-K+1}, \dots, \mathbf{s}_{t-1}, \mathbf{s}_t]$ 。与仅使用给定时间步的数据相比，这为每次在线更新提供了稍大的数据批次。由于高频下的单个时间步可能非常嘈杂，使用过去的 K 个转移有助于抑制更新。

(2) 预测模型 p_θ 将每个底层转移表示为独立的高斯分布，即

$p_\theta(\mathbf{y}_t|\mathbf{x}_t) = \prod_{t'=t-K}^{t-1} p(\mathbf{s}_{t'+1}|\mathbf{s}_{t'}, \mathbf{a}_{t'})$ ，其中每个 $p(\mathbf{s}_{t'+1}|\mathbf{s}_{t'}, \mathbf{a}_{t'})$ 由均值为 $f_\theta(\mathbf{s}_{t'}, \mathbf{a}_{t'})$ 、恒定方差为 σ^2 的高斯分布参数化。我们将该均值动力学函数 $f_\theta(\mathbf{s}_t, \mathbf{a}_t)$ 实现为一个具有三个隐藏层 (每层维度为 500) 和修正线性单元 (ReLU) 非线性激活的神经网络模型。

(3) 为了计算新的任务参数 $\theta_{t+1}(T_{\text{new}})$ (该参数可能会或可能不会被添加到任务分布 $\theta_{t+1}(T)$ 中)，我们使用一组与集合 $\mathbf{x}_t$ 分开的 K 个邻近数据点。这样做是为了避免使用评估该参数所用的同一数据集来计算参数，因为 $P_t(T_{\text{new}})$ 来自在数据 $\mathbf{x}_t$ 上评估该参数。

(4) 与标准在线流任务中下一个数据点 $\mathbf{x}_{t+1}$ 是直接给定的不同，在这种情况下，传入的数据点 (即下一个访问的状态) 受到预测模型本身的影响。这是因为，在从可能任务中选择出最可能的任务 T^* 之后，来自

控制器利用模型 $p_{\theta_{t+1}(T^*)}$ 对一系列未来动作 $\mathbf{a}_0, \dots, \mathbf{a}_H$ 进行规划，并选择能够最大化未来奖励的动作。值得注意的是，该规划过程基于随机优化，沿用了先前工作（Nagabandi 等人，2018）的方法，更多细节见附录。由于控制器的动作选择决定了下一个数据点，且控制器的选择依赖于估计的模型参数，因此在此情境下，恰当地调整模型显得尤为关键。

5) 最后，需指出的是，我们通过模型无关元学习（MAML）在元训练中获得 θ^* ，如上文方法所述。然而，在此情况下，MAML 是在基于模型的强化学习循环中执行的。换言之，在元训练的某一迭代中，模型参数被控制器用于生成在线策略轨迹，这些轨迹产生的数据随后被加入 MAML 的数据集，此过程重复直至元训练结束。

7 实验

我们实验旨在研究的问题包括：MOLe 能否 1) 在非平稳数据流中自主发现某些任务结构？2) 适应超出 k 样本学习方法所能处理范围的任务分布之外的任务？3) 识别并回退到之前见过的任务？4) 避免对近期任务过拟合，以防止下一次任务切换时性能下降？5) 优于其他方法？

为研究这些问题，我们在 MuJoCo 物理引擎（Todorov 等人，2012）中对智能体进行了实验。所用智能体为半猎豹（状态维度 21，动作维度 6）和六足爬行器（状态维度 50，动作维度 12）。利用这些智能体，我们设计了一系列具有挑战性的在线学习问题，这些问题涉及底层任务分布的多次突变和渐变，包括从先前见过的任务外推得到的任务，其中在线学习至关重要。通过这些实验，我们旨在构建能够代表真实强化学习智能体可能遇到的干扰和变化类型的问题设置。

我们在以下三节中呈现结果和分析，视频见 <https://sites.google.com/berkeley.edu/onlineviameta>。在实验中，我们与几种替代方法进行了比较，包括两种利用元训练的方法和两种未利用元训练的方法：

(a) 基于元学习的 k 样本适应：始终从元训练先验 θ^* 进行适应，这是元学习方法（Nagabandi 等人，2018）的典型做法。该方法通常不足以适应分布外更远的任务，且适应结果也不会随时间推移被保留以供未来使用。

(b) 结合元学习的持续适应：始终从前一时间步的参数出发进行梯度更新。该方法往往对近期观察到的任务过拟合，因此应表明本文方法有效识别任务结构以避免过拟合并实现召回的重要性。

(c) 基于模型的强化学习：在与上述方法相同的数据上训练一个模型，采用标准监督学习，并在整个试验过程中保持该模型固定不变（即不进行元学习，也不进行自适应）。

(d) 基于模型的强化学习与在线梯度更新：使用与基于模型的强化学习相同的模型（即无元学习），但在运行时通过梯度下降进行在线自适应。这代表了常用的动态评估方法（Rei, 2015; Krause et al., 2017; 2016; Fortunato et al., 2017）。

7.1 半猎豹（Half-Cheetah）地形坡度

本文从半猎豹（Half-Cheetah）（图 1）智能体穿越不同坡度地形的任务开始。先验模型在低幅度随机坡度地形数据上进行元训练，测试试验则在困难的外分布任务上执行，如盆地、陡坡等。如图 2 所示，基于模型的强化学习（Model-Based RL）以及带在线梯度更新的基于模型的强化学习在这些外分布任务上均表现不佳，尽管这些

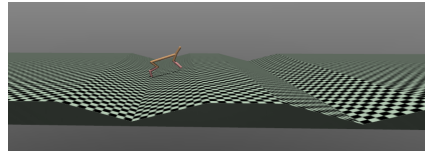


图 1：半猎豹智能体，展示其穿越带有“盆地”的地形

模型在元训练模型所接收的相同数据上进行训练。基于模型的强化学习方法性能不佳，表明需要进行模型适应（而非假设单一模型能够涵盖一切），而基于模型的强化学习结合在线梯度更新性能不佳，则表明需要元学习初始化，以实现神经网络的在线学习。

对于三种元学习与自适应方法，我们预期持续自适应结合元学习会表现不佳，因为连续的梯度步骤会导致其过拟合近期数据；也就是说，我们预期在上坡路段的经验会导致下坡路段性能下降，或出现类似情况。然而，基于定性与定量结果，我们发现元学习过程似乎已将智能体初始化在一个参数空间中，在该空间中这些不同的“任务”并未被视为有实质性差异，且通过随机梯度下降的在线学习表现良好。这表明元学习过程找到了一个任务空间，在该空间中不同坡度之间存在容易的技能迁移；因此，即使 MOLe 面临切换任务或向其动态潜在任务分布中添加新任务的选项，它也会选择不这样做（图 3）。与后文将看到的发现不同，有趣的是，此处发现的任务空间并不对应于人类可区分的类别标签。最后，需要注意的是，这些改变坡度的任务彼此并不特别相似（且所发现的任务空间确实有用），因为两个非元学习基线尽管在浅层训练坡度上具有相似的训练性能，但在这些测试任务上确实失败了。

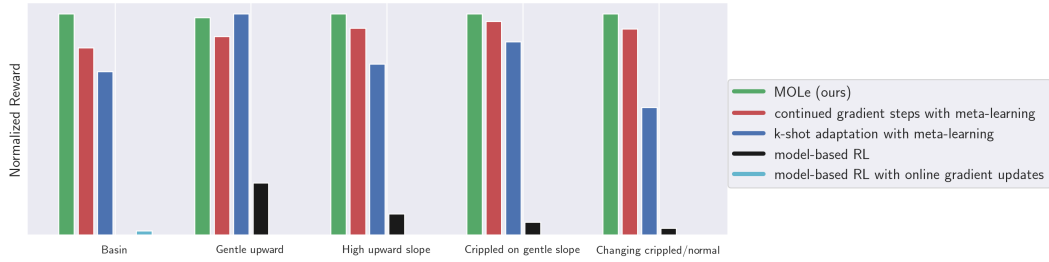


图 2：半猎豹地形遍历的结果。表现不佳的基于模型的强化学习表明单一模型并不足够，而带有在线梯度更新的基于模型的强化学习则表明元学习初始化至关重要。三种元学习方法表现相似；然而，当任务需要从先验出发进行多步梯度更新时，k-shot 自适应的性能会下降。

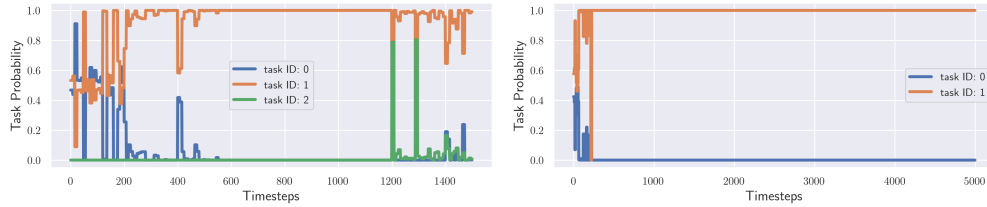


图 3：两个半猎豹地形遍历任务中潜在任务分布随时间的变化，其中每次运行中遇到的地形坡度各不相同。有趣的是，我们发现 MOLe 选择仅使用单个潜在任务变量来描述变化的地形。

7.2 半猎豹运动故障

虽然半猎豹（half-cheetah）在斜坡地形上的研究结果表明，对于表面上看似不同的任务，并不总是需要单独的任务参数，但我们同样希望研究那些在环境中经历更剧烈变化的非平稳任务分布的智能体。在这组实验中，我们在数据上训练所有模型，其中随机选择一个执行器在运行过程中发生故障。在这种情况下，故障意味着施加到该执行器的动作的极性或幅度被改变。图 4 展示了各种方法在严重超出分布的测试任务上的结果，例如同时改变所有执行器。图 4 左侧显示，当测试试验中的任务分布仅包含单一任务时，例如“符号取反”，即所有执行器被指定为相反极性，那么持续适应通过反复对传入数据进行梯度更新而表现良好。然而，如图 4 中的其他任务所示，这种持续适应的性能

当智能体经历非平稳任务分布时，性能会显著恶化。由于对最近输入数据的过度专门化，这类持续适应的方法往往会遗忘并丧失先前已有的技能。这种对过去技能的过拟合与遗忘也体现在图 4 所示的一致性能退化中。相比之下，MOLe 动态构建概率任务分布，并允许适应这些困难任务，而不会遗忘过去的技能。我们在图 5 中展示了一个示例任务设置，其中智能体经历正常与残腿操作的交替时段。该图显示了新任务和旧任务的成功识别；值得注意的是，识别和适应都是在线完成的，既没有使用过去数据存储库来执行适应，也没有人工指定的任务类别集合。

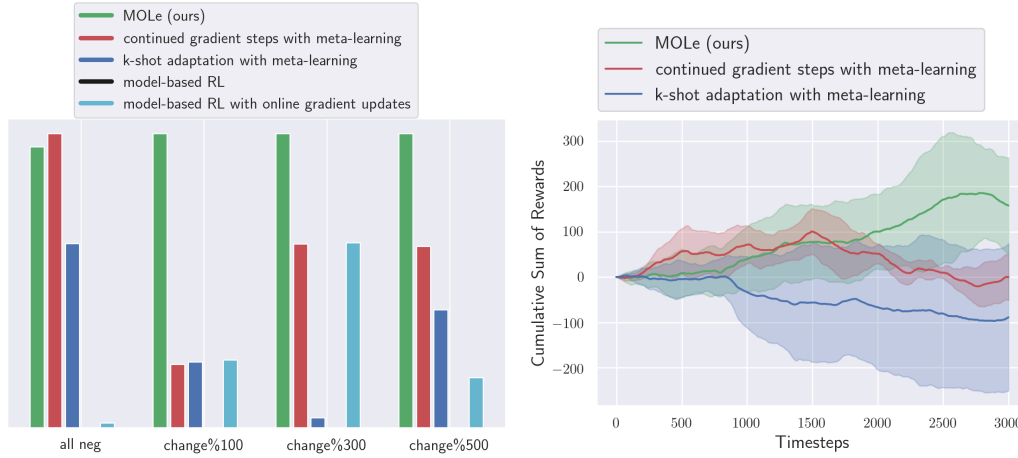


图 4：运动功能障碍试验的结果，其中不同试验展示了以不同频率调制的任务分布（或对于第一类保持恒定）。在此，在线学习对于良好性能至关重要， k 次自适应对于如此不同的任务是不充分的，而持续的梯度步骤会导致对最近所见数据的过拟合。

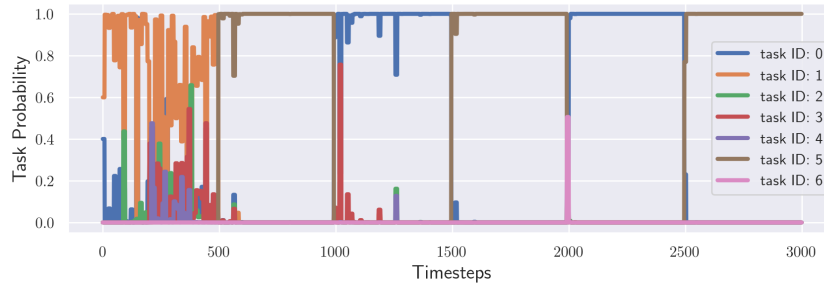


图 5：在线学习试验过程中潜在任务变量分布，其中底层运动故障每 500 个时间步改变一次。我们发现 MOLe 能够成功恢复任务结构，识别底层任务何时发生变化，并召回之前见过的任务。

7.3 六足爬行机器人末端执行器致残

为了进一步检验我们持续在线适应算法的效果，我们研究了另一个更复杂的智能体：六足爬行机器人（图 6）。在这些实验中，模型在随机关节致残（即无法施加执行器命令）的条件下进行训练。在图 7 中，我们展示了两个说明性测试任务：（1）智能体在其测试时间经历中看到一组固定的致残配置；（2）智能体在正常操作区域和腿部致残区域之间交替获得经验。

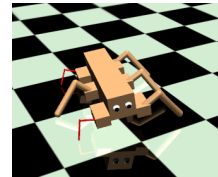


图 6：六足爬行机器人，展示腿部致残情况。

第一种设置与训练期间看到的数据相似，因此我们看到即使是基于模型的强化学习和带在线梯度更新的基于模型的强化学习基线也没有失败。然而，同时包含元学习和适应的方法确实具有更高的性能。此外，我们再次看到在单任务设置中持续梯度步骤并非有害。第二种设置的非平稳任务

分布（当腿部致残是动态的）说明了在线适应的必要性（基于模型的强化学习失败），需要良好的先验来进行适应（基于模型的强化学习配合在线梯度更新失败），过度拟合近期经验从而遗忘旧技能的有害影响（持续梯度步骤的性能低下），以及需要进一步偏离先验进行适应（k-shot 适应的性能有限）。借助 MOLe，该智能体能够构建自己的“任务”切换表示，我们看到这种切换确实对应于识别腿部致残的区域（图 7.3 左侧）。三种元学习加适应方法的累积奖励总和图（图 7.3 右侧）每 500 步都包含相同的任务切换模式：在这里，我们可以清楚地看到第 500-1000 步和第 1500-2000 步是致残区域。持续梯度步骤在第二次和第三次遇到正常操作时表现更差，而 MOLe 随着任务出现频率的增加表现明显更好。注意这两种技能的提升，其中一种技能的发展实际上并不会妨碍另一种技能。

最后，我们考察了爬行器在每次试验中经历直线行走、转弯以及有时腿部致残的实验。持续梯度步骤在“正常构型下向前行走”的前 500 个时间步的性能与 MOLe 相当（ $\pm 10\%$ 差异），但在最后 500 个时间步的性能低了 **200%**。注意，在不允许单独任务专门化/适应的情况下执行更新会产生这种不利影响。

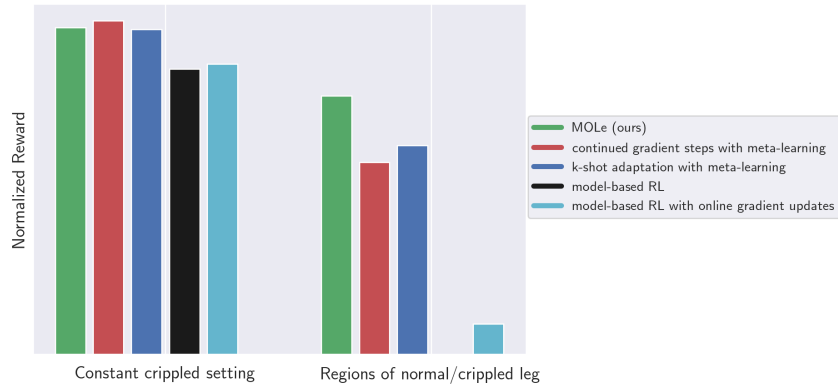


图 7：爬行器的定量结果。对于固定任务，适应并非必需，所有方法都表现良好。相比之下，当任务在试验中动态变化时，只有 MOLe 能有效地在线学习。

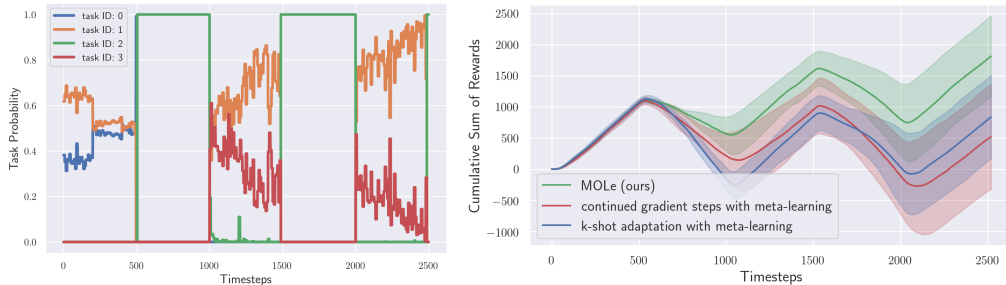


图 8：爬行器实验结果。左图：正常/致残经历交替期间潜在任务概率的在线识别。右图：MOLe 通过多次遇到相同任务而得到改进。

8 讨论

本文提出了一种面向神经网络模型的在线学习方法，该方法能够在每个试验内处理非平稳、多任务场景。我们的方法直接利用随机梯度下降（SGD）对模型进行适配，其中期望最大化（EM）算法使用中餐馆过程先验来维持任务分布并处理非平稳性。尽管在流式场景下，对于深度神经网络这类大型参数模型，随机梯度下降通常是一种较差的在线学习算法，但我们观察到，通过（1）使用模型无关元学习（MAML）对模型进行元训练以实现快速适配，以及（2）在测试时采用我们的在线算法进行概率更新，我们能够实现有效的神经网络在线学习。在实验中，我们将该方法应用于基于模型的强化学习，并证明其可用于适配面临各种新颖且意外任务的模拟机器人行为。

结果表明，我们的方法能够形成自身的任务概念，在必要时持续偏离先验（甚至学习需要更多适配的任务），并回忆之前见过的任务。虽然我们以基于模型的强化学习作为评估领域，但我们的方法具有通用性，可应用于其他流式和在线学习场景。未来工作一个令人兴奋的方向是将我们的方法应用于时间序列建模和主动在线学习等领域。

参考文献

- Maruan Al-Shedivat, Trapit Bansal, Yuri Burda, Ilya Sutskever, Igor Mordatch, and Pieter Abbeel. Continuous adaptation via meta-learning in nonstationary and competitive environments. *arXiv preprint arXiv:1710.03641*, 2017.
- Rahaf Aljundi, Punarjay Chakravarty, and Tinne Tuytelaars. Expert gate: Lifelong learning with a network of experts. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.
- Samy Bengio, Yoshua Bengio, Jocelyn Cloutier, and Jan Gecsei. On the optimization of a synaptic learning rule. In *Optimality in Artificial and Biological Neural Networks*, 1992.
- David M Blei, Andrew Y Ng, and Michael I Jordan. Latent dirichlet allocation. *Journal of machine Learning research*, 3(Jan):993–1022, 2003.
- Zdravko I Botev, Dirk P Kroese, Reuven Y Rubinstein, and Pierre LÉcuyer. The cross-entropy method for optimization. In *Handbook of statistics*, volume 31, pp. 35–59. Elsevier, 2013.
- Léon Bottou. Online learning and stochastic approximations. *On-line learning in neural networks*, 17(9):142, 1998.
- Tamara Broderick, Nicholas Boyd, Andre Wibisono, Ashia C Wilson, and Michael I Jordan. Streaming variational bayes. In *Advances in Neural Information Processing Systems*, pp. 1727–1735, 2013.
- Julien Doyon and Habib Benali. Reorganization and plasticity in the adult brain during learning of motor skills. *Current opinion in neurobiology*, 15(2):161–167, 2005.
- Yan Duan, John Schulman, Xi Chen, Peter L Bartlett, Ilya Sutskever, and Pieter Abbeel. RL2: Fast reinforcement learning via slow reinforcement learning. *arXiv:1611.02779*, 2016.
- John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 2011.
- Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. *International Conference on Machine Learning (ICML)*, 2017.
- J Randall Flanagan and Alan M Wing. Modulation of grip force with load force during point-to-point arm movements. *Experimental Brain Research*, 95(1):131–143, 1993.
- Meire Fortunato, Charles Blundell, and Oriol Vinyals. Bayesian recurrent neural networks. *arXiv preprint arXiv:1704.02798*, 2017.
- Robert Herman. *Neural control of locomotion*, volume 18. Springer, 2017.
- Matthew Hoffman, Francis R Bach, and David M Blei. Online learning for latent dirichlet allocation. In *advances in neural information processing systems*, pp. 856–864, 2010.
- Amir Jafari, Amy Greenwald, David Gondek, and Gunes Ercal. On no-regret learning, fictitious play, and nash equilibrium. In *ICML*, volume 1, pp. 226–233, 2001.
- Ghassen Jerfel, Erin Grant, Thomas L Griffiths, and Katherine Heller. Online gradient-based mixtures for transfer modulation in meta-learning. *arXiv preprint arXiv:1812.06080*, 2018.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017.

- Ben Krause, Liang Lu, Iain Murray, and Steve Renals. Multiplicative lstm for sequence modelling. *arXiv preprint arXiv:1609.07959*, 2016.
- Ben Krause, Emmanuel Kahembwe, Iain Murray, and Steve Renals. Dynamic evaluation of neural sequence models. *CoRR*, abs/1709.07432, 2017.
- David Lopez-Paz et al. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, 2017.
- Tsendsuren Munkhdalai and Hong Yu. Meta networks. *International Conference on Machine Learning (ICML)*, 2017.
- K. Murphy. *Dynamic Bayesian Networks: Representation, Inference and Learning*. PhD thesis, Dept. Computer Science, UC Berkeley, 2002. URL <https://www.cs.ubc.ca/~murphyk/Thesis/thesis.html>.
- Anusha Nagabandi, Ignasi Clavera, Simin Liu, Ronald S Fearing, Pieter Abbeel, Sergey Levine, and Chelsea Finn. Learning to adapt in dynamic, real-world environments through meta-reinforcement learning. *arXiv preprint arXiv:1803.11347*, 2018.
- Devang K Naik and RJ Mammone. Meta-neural networks that learn by learning. In *International Joint Conference on Neural Networks (IJCNN)*, 1992.
- Cuong V Nguyen, Yingzhen Li, Thang D Bui, and Richard E Turner. Variational continual learning. *arXiv:1710.10628*, 2017.
- A. Rao. A survey of numerical methods for optimal control. In *Advances in the Astronautical Sciences*, 2009.
- Sachin Ravi and Hugo Larochelle. Optimization as a model for few-shot learning. In *International Conference on Learning Representations (ICLR)*, 2017.
- Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, and Christoph H Lampert. icarl: Incremental classifier and representation learning. In *Proc. CVPR*, 2017.
- Marek Rei. Online representation learning in recurrent neural language models. *CoRR*, abs/1508.03854, 2015.
- Samuel Ritter, Jane X Wang, Zeb Kurth-Nelson, Siddhant M Jayakumar, Charles Blundell, Razvan Pascanu, and Matthew Botvinick. Been there, done that: Meta-learning with episodic recall. *arXiv preprint arXiv:1805.09692*, 2018.
- Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv:1606.04671*, 2016.
- Doyen Sahoo, Quang Pham, Jing Lu, and Steven CH Hoi. Online deep learning: Learning deep neural networks on the fly. *arXiv preprint arXiv:1711.03705*, 2017.
- Adam Santoro, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, and Timothy Lillicrap. Meta-learning with memory-augmented neural networks. In *International Conference on Machine Learning (ICML)*, 2016.
- Jurgen Schmidhuber. Evolutionary principles in self-referential learning. *Diploma thesis, Institut f. Informatik, Tech. Univ. Munich*, 1987.
- Florian Stimberg, Andreas Rutter, and Manfred Opper. Bayesian inference for change points in dynamical systems with reusable states—a chinese restaurant process approach. In *Artificial Intelligence and Statistics*, pp. 1117–1124, 2012.
- Sebastian Thrun. Lifelong learning algorithms. In *Learning to learn*. Springer, 1998.
- Sebastian Thrun and Lorien Pratt. *Learning to learn*. Springer Science & Business Media, 1998.

- Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*, pp. 5026–5033. IEEE, 2012.
- Jane X Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z Leibo, Remi Munos, Charles Blundell, Dharshan Kumaran, and Matt Botvinick. Learning to reinforcement learn. *arXiv:1611.05763*, 2016.
- Yu-Xiong Wang, Deva Ramanan, and Martial Hebert. Growing a brain: Fine-tuning by increasing model capacity. In *IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.
- Grady Williams, Andrew Aldrich, and Evangelos Theodorou. Model predictive path integral control using covariance variable importance sampling. *CoRR*, abs/1509.01149, 2015.
- Friedemann Zenke, Ben Poole, and Surya Ganguli. Continual learning through synaptic intelligence. In *International Conference on Machine Learning*, 2017.

附录 A 测试时性能与训练数据的关系

我们在下文验证，随着元训练模型使用更多数据进行训练，其在测试任务上的性能确实会提升。

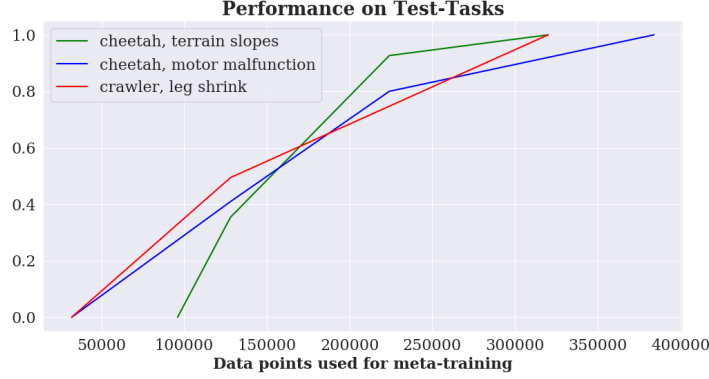


图 9：使用不同数据量进行元训练的模型在测试任务（即训练期间未见过的任务）上的性能。此处性能数值按智能体归一化，介于 0 到 1 之间。

附录 B 超参数

在所有实验中，我们使用一个由三个隐藏层组成的动力学模型，每层维度为 500，采用修正线性单元（ReLU）非线性。我们使用的控制方法是随机射击模型预测控制（MPC），在每个时间步采样 1000 个候选动作序列，每个序列的视界长度为 $H=10$ ，将其输入预测模型，并根据预期奖励进行排序。然后执行得分最高的候选动作序列中的第一个动作步骤，之后整个规划过程在下一个时间步再次重复。

下面，我们列出了实验中使用的各种方法的相关训练和测试参数。# 任务/迭代 对应于每次收集数据以训练模型时采样的任务数量，# 时间步/迭代 是该次迭代中收集的总时间步数（所有任务的总和）。

表 1：训练时超参数

	Iters	Epochs	# Tasks/itr	# TS/itr	K	outer LR	inner LR (η)
Meta-learned approaches (3)	12	50	16	2000-3000	16	0.001	0.01
Non-meta-learned approaches (2)	12	50	16	2000-3000	16	0.001	N/A

表 2：运行时超参数

	α (CRP)	LR (model update)	K (previous data)
MOLe (ours)	1	0.01	16
continued adaptation with meta-learning	N/A	0.01	16
k-shot adaptation with meta-learning	N/A	0.01	16
model-based RL	N/A	N/A	N/A
model-based RL with online gradient updates	N/A	0.01	16

C 控制器

如第 6 节所述，我们使用学习到的动力学模型结合控制器来选择要执行的下一个动作。控制器使用学习到的模型 f_θ 以及编码所需任务的奖励函数 $r(\mathbf{s}_t, \mathbf{a}_t)$ 。许多方法可用于执行此动作选择，包括交叉熵方法（CEM）（Botev 等人，2013）或模型预测路径积分控制（MPPI）（Williams 等人，2015），但在我们的实验中，我们使用随机采样射击方法（Rao，2009）。

在每个时间步 t ，我们随机生成 N 个候选动作序列，每个序列包含 H 个动作 sequence.

$$A_t^i = \{\mathbf{a}_t^i, \dots, \mathbf{a}_{t+H}^i\} \quad (10)$$

然后，我们使用学习到的动力学模型来预测执行这些候选动作序列所产生的状态 action sequences.

$S^i = \{\hat{\mathbf{s}}_{t+1}^i, \dots, \hat{\mathbf{s}}_{t+H+1}^i\}$ 其中 $\hat{\mathbf{s}}_{p+1}^i = f_\theta(\hat{\mathbf{s}}_p^i, \mathbf{a}_p^i)$ 接下来，我们使用奖励函数选择具有最高关联预测奖励的动作序列。

$$i^* = \operatorname{argmax}_i \sum_{t'=t}^{t'+H} r(\hat{\mathbf{s}}_{t'}^i, \mathbf{a}_{t'}^i) \quad (12)$$

接下来，本文并非执行所选最优动作的整个序列 $A_t^{i^*}$ ，而是采用模型预测控制（Model Predictive Control, MPC）框架，仅从当前状态 $\mathbf{s}_t$ 执行第一个动作 $\mathbf{a}_t^{i^*}$ 。随后在下一时间步重新规划；这种模型预测控制的使用方式通过在每个时间步利用更新后的状态信息重新规划，能够防止误差累积，从而补偿模型的不准确性。